SimpleDB+ join experiments

Larger-dataset analysis after the external-sort fixes. Both experiments use the same synthetic student-schema database and were measured locally on one machine; no teammate measurements are implied.

1. Dataset and experimental setup

Table	Tuples	Purpose
bench_student	1000	Student details
bench_enroll	4000	Student-section enrollments
bench_section	200	Course sections
bench_course	50	Courses
bench_dept	10	Departments; not joined in these experiments

The separate database /tmp/simpledb-scaled-1000 leaves the original studentdb untouched. Physical insertion order was shuffled with seed 20261018. An index on bench_enroll(studentid) supports Experiment 1. Both queries return exactly 4,000 rows.

Data distribution. Each student has four enrollments; each section has 20 enrollments; each course has four sections. The generator maps all four enrollments of a student to the same section. This deliberately repetitive, uniform data exercises duplicate join keys but does not represent realistic course-selection diversity or skew.

Representative tuples

Examples below follow the generator rules; they are not physical scan order. Column order is stated explicitly.

```
student (sid, sname, majorid, gradyear): (1, s1, 1, 2021)
enroll  (eid, studentid, sectionid, grade): (1, 1, 1, B+)
                                            (1001, 1, 1, B+)
section (sectid, courseid, prof, yearoffered): (1, 1, p1, 2021)
course  (cid, title, deptid): (1, course1, 1)
dept    (did, dname): (1, dept1)
```

Measurement protocol

Platform: macOS 15.7, aarch64, OpenJDK 17.0.14, eight reported processors. Database block size: 400 bytes; buffer pool: eight frames. JVM: -Xms256m -Xmx512m. One JVM ran 20 configurations in seeded shuffled order, each with one discarded warmup and three measured executions (60 measured executions total).

System.nanoTime measured plan opening, full result consumption/serialization and scan closing. Sorting, partitioning and temporary-table work are included; planning, database copying/recovery, validation, console output and rollback are excluded. Each configuration used an isolated database copy. Every execution was checked against an independent result multiset and for buffer leaks.

2. Experiment 1: two-table join

```
select sid, sname, eid, grade
from bench_student, bench_enroll
where sid = studentid
```

All five configurations use student as the left/outer input and enroll as the right input, followed by projection onto the same four fields. The runner enforces the requested method rather than silently substituting another algorithm.

Algorithm	Median ms	Minimum ms	Maximum ms
Index nested loop	31.719	30.355	31.867
Partitioned hash	260.187	256.823	268.377
Sort-merge	822.269	809.850	1,248.254
Multi-buffer product + filter	2,260.452	2,217.027	2,352.331
Tuple nested loop	2,637.904	2,618.795	3,024.162

Plans and results

Index join probes the enrollment studentid index for each student. Hash join partitions the inputs and streams matching pairs. Merge join sorts both join keys and merges them, replaying duplicate groups. Tuple nested loop repeatedly scans the right input. Multi-buffer execution forms chunked products and applies the equality filter.

sid	sname	eid	grade
1	s1	1	B+
2	s2	2	B
3	s3	3	C
1	s1	1001	B+

Representative result tuples, shown by enrollment ID. All methods returned the same 4,000 tuples; output order is unspecified because the query has no ORDER BY.

Discussion

Index join was fastest at **31.719 ms**. The existing lookup index avoids scanning all enrollment rows for every student. Hash join took 260.187 ms, approximately 10.1 times faster than tuple nested loop. This larger workload exposes the repeated-scan cost that was insignificant on the original nine-student database.

Sort-merge took 822.269 ms: faster than nested loop, but slower than hash and index join. It still pays for sorting unsorted input. Relative to the earlier implementation on the same dataset, its median fell from 3,868.230 ms to 822.269 ms (about 4.7 times faster). These were separate sessions, so the ratio is descriptive rather than a controlled causal estimate.

3. Experiment 2: four-table join

```
select sid, sname, eid, title, prof, grade
from bench_student, bench_enroll, bench_section, bench_course
where sid = studentid and sectionid = sectid
  and courseid = cid
```

S = student, E = enroll, T = section, C = course (all with the bench_ prefix). The three fixed plans are left-deep; each applies the relevant equality as tables become available. Every plan projects the same six fields and returns 4,000 rows.

Plan	Join order	Actual intermediate row counts
P1	(((S join E) join T) join C)	4,000 -> 4,000 -> 4,000
P2	(((C join T) join E) join S)	200 -> 4,000 -> 4,000
P3	(((T join E) join S) join C)	4,000 -> 4,000 -> 4,000

Intermediate counts follow the generated key relationships, not EXPLAIN estimates. Each fixed order is executed under four method restrictions, using that method at all three joins.

Plan	Nested loop	Hash	Sort-merge	Multi-buffer
P1	3,419.603	1,363.380	5,266.237	3,904.256
P2	3,600.890	861.435	3,250.910	4,488.017
P3	3,704.953	1,313.860	5,180.828	3,885.665

Entries are median milliseconds (three measurements each). Hash wins for all three fixed orders. P2 hash is fastest at 861.435 ms, about 1.58 times faster than P1 hash. P2 begins with only 200 joined rows; intermediate widths and repeated work also differ, so cardinality alone is not a complete explanation.

Merge is faster than nested loop for P2, but slower for P1 and P3. Nested loop is fastest among its own variants with P1; multi-buffer is fastest with P3. There is therefore no single best join order independent of the algorithm.

Optimizer-selected order under method restrictions

Only allowed method	Actual order	Median ms
Tuple nested loop	C -> T -> E -> S	3,607.623
Partitioned hash	C -> T -> E -> S	912.695
Sort-merge	C -> T -> E -> S	3,532.947

These additional runs leave table ordering to the heuristic optimizer. Even equivalent physical plans may time differently because they execute at different points in the session. Index-only four-table plans are unavailable with the existing index set; no extra indexes were added to manufacture such a comparison.

Example output: (1, s1, 1, course1, p1, B+) and (1, s1, 1001, course1, p1, B+). Repeated student/course values are legitimate because each row represents a distinct enrollment.

4. Interpretation, limitations and reproducibility

What changed in the implementation

The sort cursor now saves/restores all run positions, the active run and exhaustion state, preventing incorrect duplicate-key merge results. External sorting creates block-budgeted in-memory initial runs and uses priority-queue k-way merging. Merge joins cap final sorted cursors at two runs each to leave buffer space for sibling operators; intermediate passes remain k-way.

SortPlan now includes estimated sorting I/O, and TablePlanner no longer adds a separate local correction. These estimates count modeled block work, not elapsed time. Existing cardinality guesses and repeated predicate selectivity can still underestimate intermediate rows, even to zero; the benchmark checks actual result multisets independently.

Merge configuration	Earlier median ms	Current median ms
Two-table	3,868.230	822.269
Four-table P1	6,633.090	5,266.237
Four-table P2	11,286.989	3,250.910
Four-table P3	11,967.709	5,180.828

Limits on the conclusions

Only three measurements per configuration and one JVM were used for this larger dataset. OS caches were not flushed, and system load, JIT compilation and garbage collection were not controlled. We report medians and preserve all samples, but do not claim statistical significance or cold-disk performance. Before/after runs also occurred in separate sessions.

The data is synthetic, uniform and correlated: a student repeats the same section in four enrollment rows. Results do not establish performance under skew or realistic enrollment diversity. Hash partition tables use Java heap outside the eight database buffer frames; sort run payloads are bounded in block equivalents, with Java object overhead additional. This compares these implementations, not algorithms under a strict equal-memory budget.

Conclusion. On this dataset, index join is best for the two-table query and P2 hash is best among the three fixed four-table plans. Improved sorting makes merge substantially more competitive, but it does not win universally. Both algorithm and join order matter; the original tiny-data ranking should not be generalized to larger inputs.

Evidence and reproduction

Measured code: b605f19 on justin/join-experiments; sorting fix on Lab 6: 7316d23. Raw results were committed in 1cdd9d1. Source files: output/joins-scaled-sort-fixed/fork-1/{samples.csv, plans.txt, environment.txt}; prior comparison: output/joins-scaled/fork-1/samples.csv. Exact trees and per-run durations accompany this report.

```
java -Xms256m -Xmx512m -cp <compiled-classes> \
  simpledb.test.JoinExperiments /tmp/simpledb-scaled-1000 \
  output/joins-scaled-new/fork-1 1 3 20261018 bench_
```

Use a fresh output directory. To recreate the database, run CreateJoinBenchmarkDB with a new directory followed by: 1000 4000 200 50 20261018. These are measurements from one machine; separately label any additional teammate results rather than presenting them as already collected.